

THE PYTHON LOOPS TRAINER

or: How I Learned to Stop Worrying and Loop the Loop

A field guide to going in circles, on purpose

What's inside this thrilling page-turner:

```
for loops ..... walking a line of things
nested loops ..... loops inside loops inside regret
while loops ..... looping until you say when
faux do-while ..... the loop Python pretends it doesn't have
break ..... the emergency exit
continue ..... the 'skip this one' button
try / except ..... catching the wreckage gracefully
```

WARNING

Contains at least one infinite loop. If you are still reading this in the year 3000, you forgot your `i += 1`. Please press the stop button.

*Master of Data Science - CMPINF-2100 - Module 2 Companion Notes
CompSci-1 reading level. No prior loop trauma assumed.*

Table of Contents

- 0. How to Read This Thing** - the ground rules
- 1. The for Loop** - walking a line of things
- 2. range() and len()** - making for loops count
- 3. Nested Loops** - the odometer principle
- 4. The while Loop** - looping until you say when
- 5. The Faux Do-While** - run first, ask questions later
- 6. break** - the emergency exit
- 7. continue** - the skip button
- 8. break + continue Together** - ordering matters
- 9. try / except** - catching the wreckage
- 10. The Cheat Sheet** - everything on one page

Each section follows the same rhythm: the idea in plain words, the syntax, a worked example you can trace by hand, and the trap that will bite you. Read the traps twice. The traps are where the points live.

0. How to Read This Thing

A loop is a way to tell the computer 'do this again' without writing it out again and again. That is the whole idea. Everything else is bookkeeping: which thing to do it to, how many times, and when to quit.

Python gives you two real loops, the **for** loop and the **while** loop, plus three little tools that steer them: **break**, **continue**, and the **try / except** safety net. Master those five pieces and you can build any looping behavior, including the do-while loop that Python technically does not ship with.

Three symbols you'll see in the margins of your brain

The Idea plain-English explanation, no jargon.

The Trace a step-by-step table showing what the loop actually does, one pass at a time.

The Trap the specific way this construct breaks. Usually an infinite loop or an off-by-one.

One promise: every code sample here runs. If you type it into a Jupyter cell exactly as written, it works. The comments marked with # are notes to you, not code that does anything.

1. The for Loop

The Idea

A Python **for** loop walks through a collection and hands you one item at a time. It is not a counter. It does not count up from zero unless you ask it to. It simply says: 'for each thing in this group, do the following.' When the group runs out, the loop stops on its own. You never have to tell it when to quit.

If you come from C, C#, or JavaScript, this is the *for-each* loop, not the `for (i=0; i<n; i++)` counting loop. Python only has the for-each kind. That counting loop you remember is built separately, using `range()`, which is the next section.

The Syntax

```
fruit = ['apple', 'orange', 'banana']

for item in fruit:          # 'item' becomes each fruit in turn
    print(item)             # the indented block runs once per fruit

# Output:
# apple
# orange
# banana
```

Read it out loud: 'for each item in fruit, print item.' The loop variable (here `item`) is yours to name. On pass one it holds 'apple', on pass two 'orange', and so on. Convention: name it singular, because it holds one thing per pass.

Why it can't run forever

A for loop iterates a **finite** collection. Three fruits means three passes, full stop. There is no condition for you to get wrong and no counter for you to forget. This safety is the main reason to prefer for over while whenever you are walking a known group of things.

2. range() and len()

The Idea

Sometimes you do not have a list to walk; you just want to do something a set number of times, or you want the position numbers (0, 1, 2, ...) rather than the items. That is what **range()** is for. It manufactures a sequence of numbers for the for loop to walk.

len() is simpler: it tells you how many items are in something. `len(fruit)` is 3. It returns a single number you can use anywhere.

range has three dials: start, stop, step

```
range(5)           # 0, 1, 2, 3, 4      (stop only; starts at 0)
range(2, 5)        # 2, 3, 4          (start, stop)
range(0, 10, 2)     # 0, 2, 4, 6, 8    (start, stop, step)

for i in range(5):
    print(i)        # prints 0 1 2 3 4 on separate lines
```

The exclusive-stop rule (memorize this one)

range **stops before** the stop number. `range(5)` gives you five numbers: 0 through 4, NOT through 5. This is the same rule as slicing and it is the source of most off-by-one bugs. Five numbers, last one is four. Always.

Walking a list by index (the long way)

You can combine them to walk a list by position. `range(len(list))` produces exactly the valid indices, because `len` gives the count and `range` stops one short of it, which lines up perfectly with zero-based indexing.

```
masters = ['HeMan', 'Teela', 'Orko', 'ManAtArms']

for i in range(len(masters)):    # i = 0, 1, 2, 3
    print(i, masters[i])        # use i to reach into the list

# 0 HeMan
# 1 Teela
# 2 Orko
# 3 ManAtArms
```

The Trap: range(list) is not a thing

You must hand range a **number**, never a raw list. `range(masters)` crashes. It has to be `range(len(masters))`. And honestly, if you only want the items and not their positions, skip all this and just write `for m in masters:`. The `range(len(...))` form is only worth it when you genuinely need the index number `i`.

Bonus: printing a range looks weird on purpose

`print(range(0, 10, 2))` shows you `range(0, 10, 2)`, not the numbers. A range is 'lazy': it stores the recipe, not the baked numbers. To see the actual numbers, wrap it: `list(range(0, 10, 2))` gives `[0, 2, 4, 6, 8]`.

3. Nested Loops

The Idea

Put a loop inside another loop and you get a **nested loop**. The rule is simple but powerful: the inner loop runs **completely**, start to finish, for every single pass of the outer loop. One step outside, a full lap inside; another step outside, another full lap inside.

The Odometer Principle

Think of a car odometer. The rightmost wheel (the inner loop) spins fastest and completes a full turn before the wheel to its left (the outer loop) clicks forward by one. The deeper the loop, the faster it spins.

```

for outer in range(2):          # the slow wheel
    for inner in range(3):      # the fast wheel: full spin each outer step
        print(outer, inner)

# 0 0      <- outer=0, inner spins all the way...
# 0 1
# 0 2
# 1 0      <- ...then outer clicks to 1, inner spins again
# 1 1
# 1 2

```

Two outer passes, three inner passes each, equals six lines total. That multiplication is the thing to respect.

The Trap: nesting multiplies, it does not add

Total iterations = outer count **times** inner count, not plus. Two loops of 100 is 10,000 passes. Three nested loops of 100 is one **million** passes. A fourth level and you may be waiting a while. Nesting is fine and necessary (it is how you walk 2D and 3D data), but always know your multiplication before you hit Run.

Where you'll actually use this

A spreadsheet or image is a grid: rows and columns. To visit every cell, you loop rows on the outside and columns on the inside. Three nested loops walk a 3D block (layers, rows, columns), which is exactly how you will traverse NumPy arrays later in the program.

4. The while Loop

The Idea

A **while** loop repeats as long as a condition stays true. Unlike the for loop, it has no collection to exhaust, so it does **not** stop on its own. **You** are responsible for eventually making the condition false. Forget to, and it runs forever.

The three required parts

```
i = 0                # 1. INITIALISE: set up the variable
while i < 5:          # 2. CONDITION: loop runs while this is true
    print(i)
    i += 1           # 3. ADVANCE: nudge the variable toward false

# prints 0 1 2 3 4, then 5 < 5 is false, so it stops
```

Miss any of the three and the loop misbehaves. Skip the initialise and the variable does not exist. Skip the condition and it is not a while loop. Skip the advance and you have built the classic infinite loop.

Trace it by hand

i	i < 5 ?	action	i after
0	True	print 0	1
1	True	print 1	2
2	True	print 2	3
3	True	print 3	4
4	True	print 4	5
5	False	stop	-

The Trap: the infinite loop

If you forget the advance step (the `i += 1`), `i` stays 0 forever, `0 < 5` is always true, and the loop never ends. In Jupyter this freezes the cell. The fix is the square stop button at the top of the notebook. Know where it is before you write your first while loop, because you **will** meet this.

for vs while: which to reach for

Use for when you know the count or are walking a known group: do this N times, or visit every item. It cannot infinite-loop.

Use while when the number of repetitions depends on something you cannot predict: loop until the user quits, until a value converges, until input is valid. The count is unknown, so the loop is condition-driven.

Rule of thumb: if you catch yourself writing `i = 0` before the loop and `i += 1` inside it just to count, you probably wanted a for loop instead.

5. The Faux Do-While

The Idea

A do-while loop (which C and C# have) runs the body **once first**, then checks the condition. It guarantees at least one pass. Python has no do-while keyword, so you fake it. You need this whenever the thing you want to test does not exist until the body has run at least once: you cannot check a user's answer before you have asked the question.

Pattern 1: while True + break (the popular one)

Make the loop infinite on purpose, then put the real test at the **bottom** as an if-break. Because the test is at the end, the body always runs once before the test is ever reached.

```
while True:                                # loop forever...
    answer = input('Name? ')                # ...so the body always runs once
    if answer != '':                        # test at the BOTTOM
        break                               # good input: leave the loop
    print('Blank, try again.')              # otherwise it loops back around

print('Hello,', answer)
```

Pattern 2: the primed sentinel

Keep a normal top-tested while, but pre-set the variable to a fake value that guarantees the first check passes. After the first real pass it holds a genuine value and behaves normally.

```
import random

roll = 0                                    # fake value: 0 is never a real die roll
while roll != 6:                            # 0 != 6 is true, so we get in the door
    roll = random.randint(1, 6)
    print('Rolled', roll)

print('Got a 6!')
```

The Trap: while True with no break is forever

Pattern 1 leans on the break to escape. If your break condition can never become true, you have written a permanent infinite loop with extra steps. Always be able to point at the exact line that gets you out. With Pattern 2, the trap is forgetting to update the primed variable inside the loop, which strands it at the fake value.

When this is genuinely useful

Input validation (ask, then check), retry-until-success (try, then see if it worked), and menus (show, then read the choice). In all three the first action has to happen before you have anything to test. That is the do-while shape.

6. break

The Idea

break leaves the loop immediately and entirely. The instant Python reaches it, the loop is over: no more passes, control jumps to whatever comes after the loop. It is the emergency exit. The most common use is 'I found what I was looking for, stop searching.'

```
for n in [1, 2, 3, 4, 5]:
    if n == 3:
        break          # the moment n is 3, the whole loop ends
    print(n)

# prints 1, then 2, then hits break at 3 and stops.
# 4 and 5 are never looked at.
```

n	n == 3 ?	result
1	no	print 1
2	no	print 2
3	yes	break: loop ends
4	-	never reached
5	-	never reached

The Trap: break only escapes ONE loop

In nested loops, a break inside the inner loop exits only the inner loop. The outer loop keeps right on going. People expect break to blast out of all levels at once; it does not. Escaping several levels takes a flag variable or moving the code into a function and using return.

Bare break at the bottom = a one-pass loop

If you put an unconditional break at the end of a loop body, the loop runs exactly once and quits. That makes the loop pointless (a plain if would do the same). break is almost always meant to be guarded by an if, so it fires only in a specific situation.

7. continue

The Idea

continue skips the rest of the current pass and jumps to the next one. The loop does **not** end; it just abandons the leftover work for this one item and moves on. It is the skip button: 'this item does not qualify, on to the next.'

```
for n in [1, 2, 3, 4, 5]:
    if n == 3:
        continue      # skip ONLY this pass
    print(n)

# prints 1, 2, 4, 5.
# 3 is skipped, but the loop keeps going.
```

n	n == 3 ?	result
1	no	print 1
2	no	print 2
3	yes	continue: skip, no print
4	no	print 4
5	no	print 5

break vs continue in one breath

break ends the race. **continue** skips one runner and keeps the race going. Same setup, n == 3: break prints 1, 2. continue prints 1, 2, 4, 5.

The Trap: continue can strand a while loop

In a while loop, if your advance step (i += 1) sits **below** the continue, the continue jumps over it. The counter never moves, the condition never changes, and you have an infinite loop. In a for loop this cannot happen because the loop advances automatically. So with while + continue, put the advance **before** any continue.

```

i = 0
while i < 5:
    if i == 2:
        continue          # BUG: jumps over i += 1, so i is stuck at 2 forever
    print(i)
    i += 1                 # this line never runs when i == 2

```

8. break + continue Together

The Idea

When both steering tools appear in the same loop body, they are checked top to bottom, just like an if/elif chain. An earlier continue can bounce an item out before a later break ever gets a chance to look at it. **Order changes behavior.**

```

numbers = [1, 2, 3, 4, 5, 6, 7, 8]

for n in numbers:
    if n % 2 != 0:          # STAGE 1: odd? skip it
        continue
    if n > 6:               # STAGE 2: past 6? bail out entirely
        break
    print(n)               # STAGE 3: survivors get printed

# Output: 2, 4, 6

```

Trace the survivors

n	stage 1: odd?	stage 2: > 6?	result
1	odd: continue	-	skipped
2	even: pass	no: pass	print 2
3	odd: continue	-	skipped
4	even: pass	no: pass	print 4
5	odd: continue	-	skipped
6	even: pass	no: pass	print 6
7	odd: continue	never tested	skipped
8	even: pass	yes: break	loop ends

The subtle bit: 7 never meets the break

The break is meant to fire when $n > 6$. The first number over 6 is 7, but 7 is odd, so it hits continue at stage 1 and bounces before the break is ever evaluated. The loop actually ends at 8, the first **even** number over 6. The lesson: an earlier continue can hide a later break from some items entirely. The order you stack them in is part of the logic, not decoration.

9. try / except

The Idea

Some operations can fail at runtime: dividing by zero, turning the word 'cat' into a number, reading a missing file. Without protection, a failure crashes the whole program. **try / except** lets you attempt the risky code and, if it blows up, handle the failure gracefully instead of crashing.

If you know try/catch from other languages, this is the same idea with different words. Python spells 'catch' as **except** and 'throw' as **raise**.

Other languages	Python	what it does
try	try	wrap the risky code
catch	except	handle a failure
throw	raise	trigger a failure yourself
finally	finally	always runs, cleanup

The four parts

```
try:
    x = int(input('Number? '))    # might fail (ValueError)
    result = 10 / x               # might fail (ZeroDivisionError)
except ValueError:
    print('That was not a number.')    # runs if int() failed
except ZeroDivisionError:
    print('Cannot divide by zero.')    # runs if the divide failed
else:
    print('All good, result is', result) # runs ONLY if nothing failed
finally:
    print('Done either way.')         # ALWAYS runs, success or not
```

You can stack several except blocks for different failure types; they are checked top to bottom, first match wins, like an if/elif chain. The **else** runs only when the try block succeeded with no error. The **finally** runs no matter what, which makes it the place for cleanup.

raise: throwing your own error

`raise ValueError('Age cannot be negative')` deliberately triggers a failure. It travels up looking for a matching `except`, exactly like a failure Python raised on its own. Use it to reject bad input on purpose.

The Trap: the bare except that swallows everything

Writing `except:` with no type, or `except: pass`, catches **every** error and hides it, including your own typos and logic bugs. Your program stops crashing but starts producing silently wrong answers, which is worse. Always name the specific error you expect (`except ValueError:`) and let the unexpected ones crash loudly so you can see and fix them. A loud crash is information; a swallowed one is a landmine.

The data-science payoff: `try + continue`

Real data is messy. The everyday pattern is: loop over the rows, try to process each one, except the broken ones, report them, and continue so a single bad row does not kill your whole analysis.

```
for row in data:
    try:
        value = float(row)          # the risky bit
    except ValueError:              # only the failure we expect
        print('Skipped bad row:', row)
        continue                   # report it, keep going
    process(value)
```

10. The Cheat Sheet

Everything above, compressed. If you only reread one page before a quiz, this is the page.

Tool	One-line job	Watch out for
for	walk a known group, do this N times	use it whenever you can; it can't loop forever
range()	make numbers for a for loop to walk	stops BEFORE the stop value; needs a number, not a list
len()	how many items are in something	rides along with range; never drives a loop alone
nested for	walk a grid (rows x columns)	iterations multiply; three deep gets huge fast
while	loop until a condition flips false	YOU must make it stop; forgetting = infinite loop
faux do-while	run once, then decide (while True + break)	make sure a break can actually be reached
break	leave the loop right now	escapes only the innermost loop
continue	skip this pass, keep looping	in while, can jump over your <code>i += 1</code>
try/except	attempt risky code, handle failure	never use bare except; name the error
raise	trigger a failure on purpose	it crashes the program if nothing catches it

The five rules that prevent 90% of loop bugs

1. Prefer **for** over while whenever you are walking a known group.
2. In a **while** loop, always include the advance step that moves toward stopping.
3. Remember **range stops one short** of its stop value.
4. **break** exits one loop; **continue** skips one pass. Don't mix them up.
5. Catch **specific** errors by name; never write a bare except that swallows everything.

That's the whole loop toolkit. Five pieces (for, while, break, continue, try/except) plus two helpers (range, len). Everything fancier you ever build with loops is just these parts arranged with a little imagination. Now go forth and iterate responsibly. And for the last time: remember your `i += 1`.